Machine Vision - Assignment 2

Supankan (Index: TBA)

GitHub Repository: `https://github.com/Supankan/Machine-Vision---Assignment-2`

March 2, 2026

Introduction

This report details the approaches and methodologies used to solve Assignment 2 for the Machine Vision (IN4640) module. It covers total least squares, RANSAC multi-line fitting, pinhole camera dimensions, and homography applications using OpenCV. Source code and materials are hosted on the GitHub repository linked above.

Question 1

We are given a dataset in `lines.csv` which represents three point scatters conforming to three lines. The first five rows reveal that the coordinates of the points for the three lines are organized into columns $(x_1, x_2, x_3, y_1, y_2, y_3)$.

(a) Total Least Squares on the First Line

To fit a line strictly using the data corresponding to the first line (the x_1 and y_1 columns), we perform Total Least Squares (TLS). This involves centralizing the coordinates by subtracting their mean and then performing Principal Component Analysis (PCA) or Singular Value Decomposition (SVD) on the centered data matrix. The singular vector associated with the smallest singular value provides the optimal normal vector (a, b) of the fitted line $ax + by + d = 0$.

By computing this on the first line's data, we obtain the following parameters for the line equation:

$$\begin{aligned} a &= 0.7736 \\ b &= -0.6337 \\ d &= -3.7942 \end{aligned}$$

This defines the line:

$$0.7736x - 0.6337y - 3.7942 = 0$$

Or in the standard $y = mx + c$ format:

$$y = 1.2207x - 5.9872$$

(b) Fitting Three Lines using RANSAC

Now, treating all points collectively as a single dataset (by flattening all X columns and Y columns), we run the RANSAC (Random Sample Consensus) algorithm to iteratively find the three lines. The process is iteratively implemented in `solve_q1.py` as follows:

```
def ransac_line_fit(points, n_iterations=1000, threshold=0.1):
    best_line = None
    best_inliers = []

    n_points = len(points)

    for _ in range(n_iterations):
        # Sample 2 distinct points
        idx = np.random.choice(n_points, 2, replace=False)
        p1, p2 = points[idx]

        # Line equation ax + by + d = 0
        diff = p2 - p1
        if np.linalg.norm(diff) < 1e-6: continue

        a, b = -diff[1], diff[0]
        norm = np.hypot(a, b)
        a, b = a / norm, b / norm
        d = -(a * p1[0] + b * p1[1])

        # distance of points to the line
        distances = np.abs(a * points[:, 0] + b * points[:, 1] +
           d)
        inlier_idx = np.where(distances < threshold)[0]

        if len(inlier_idx) > len(best_inliers):
            best_inliers = inlier_idx
            best_line = (a, b, d)

    return best_line, best_inliers
```

This algorithm handles iterating through the parameters:

1. Randomly sample two distinct points to form a line hypothesis.
2. Calculate the perpendicular distance of all available points to this line.
3. Count the number of inliers (points within a specified distance threshold).
4. Retain the line with the largest consensus set (most inliers).
5. Mask (remove) the winning inliers from the dataset.
6. Repeat the process on the remaining points to discover the next line.

After running this iterative process (with an inlier threshold of 0.2, and roughly 2000 iterations per line), and refining each consensus set with TLS, the resulting parameters for the three discovered lines are:

Line 1:

- Parameters: $a = -0.7284$, $b = 0.6851$, $d = -0.6595$
- Equation: $-0.7284x + 0.6851y - 0.6595 = 0 \implies y = 1.0631x + 0.9625$
- Inlier count: 43 points

Line 2:

- Parameters: $a = -0.3801$, $b = -0.9250$, $d = 2.0074$
- Equation: $-0.3801x - 0.9250y + 2.0074 = 0 \implies y = -0.4109x + 2.1703$
- Inlier count: 39 points

Line 3:

- Parameters: $a = 0.7894$, $b = -0.6139$, $d = -3.5935$
- Equation: $0.7894x - 0.6139y - 3.5935 = 0 \implies y = 1.2859x - 5.8537$
- Inlier count: 36 points

Note: As RANSAC is a randomized algorithm, the exact parameters and ordering of the extracted lines may vary slightly across different runs depending on the initialization and chosen threshold.

Question 2

We are tasked with finding the physical sizes of a pair of earrings shown in an image, given the following camera parameters:

- Focal length (f): 8 mm
- Distance to object (Z): 720 mm
- Pixel size: $2.2\,\mu\text{m} = 0.0022$ mm

Assuming a standard pinhole camera model where the optical axis is perpendicular to the imaging plane (so the object is fronto-parallel), the magnification M is given by:

$$M = \frac{f}{Z}$$

The physical size of the object X relates to its size on the sensor x as:

$$X = x \cdot \frac{Z}{f}$$

Since the sensor size x can be expressed in terms of the number of pixels spanning the object and the pixel size:

$$X = (\text{pixels} \times \text{pixel_size}) \cdot \frac{Z}{f}$$

By writing a Python script to analyze the image `earrings.jpg`, we threshold the image to find the bounding boxes of the two earrings. The script confirms both earrings span a bounding box of 382×400 pixels.

Figure 1: Detected bounding boxes for Earrings

Applying the formula:

$$\text{Width} = (382 \times 0.0022 \text{ mm}) \times \frac{720 \text{ mm}}{8 \text{ mm}} = 0.8404 \text{ mm} \times 90 = 75.64 \text{ mm}$$
$$\text{Height} = (400 \times 0.0022 \text{ mm}) \times \frac{720 \text{ mm}}{8 \text{ mm}} = 0.8800 \text{ mm} \times 90 = 79.20 \text{ mm}$$

Thus, the physical dimensions of each earring are approximately **75.64 mm** $\times$ **79.20 mm**.

Question 3

To superimpose a country flag onto the cricket turf shown in the image, we apply a planar homography mapping the rectangular corners of the 2D flag image to the perspective-distorted four corners of the turf.

The program calculates the transform and blends it seamlessly into the pitch:

```
        # Destination points (the user's clicks)
        pts_dst = np.array(points, dtype=float)

        # Source points (corners of the flag image)
```

```
        h, w = flag_img.shape[:2]
        pts_src = np.array([[0, 0], [w - 1, 0], [w - 1, h - 1],
            [0, h - 1]], dtype=float)

        # Calculate Homography
        H, status = cv2.findHomography(pts_src, pts_dst)

        # Warp source image to destination based on homography
        temp_flag_warped = cv2.warpPerspective(flag_img, H, (img.
            shape[1], img.shape[0]))
```

Figure 2: Superimposed country flag (Sri Lanka) on the cricket turf using Homography.